Comparing Fuzz4All Performance with Newer Models

CS 6501: Software Security Testing Project

Eric Weng

Overview

- Relevant Paper: *Fuzz4All: Universal Fuzzing with Large Language Models*
- Authors: Xia et al., 2024
- Paper Idea: Automatically fuzz any program with LLMs-generated fuzzing inputs
- Project Idea: Evaluate Fuzz4All coverage performance with newer LLMs

Compiler Fuzzing

- Fuzzing a compiler for a language requires deep domain expertise
- Mutational fuzzers are unaware of target's semantics
- Generational fuzzers only target a small subset of grammar
- New seeds only go so far before plateauing
- Languages evolve faster than fuzzers

Example: Hello World

```java
// Java
public class Main {
  public static void main(String[] args) {
    System.out.println("Hello, world!");
  }
}
```

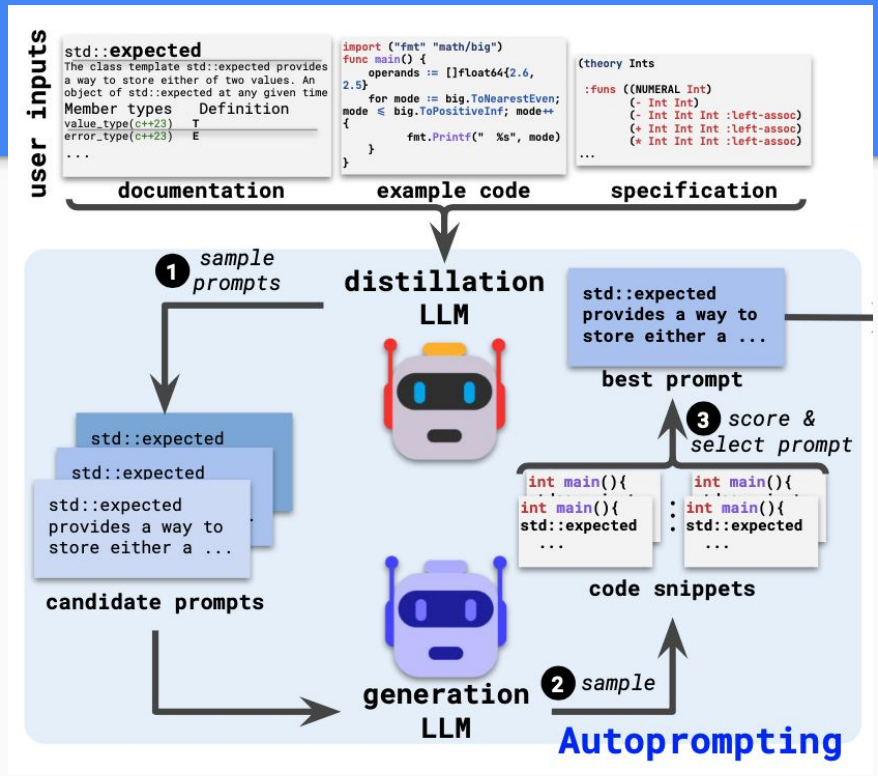
```c
// C
#include <stdio.h>

int main() {
  printf("%s", "Hello, world!");
  return 0;
}
```

- Now consider: arrays, methods, objects, pointers

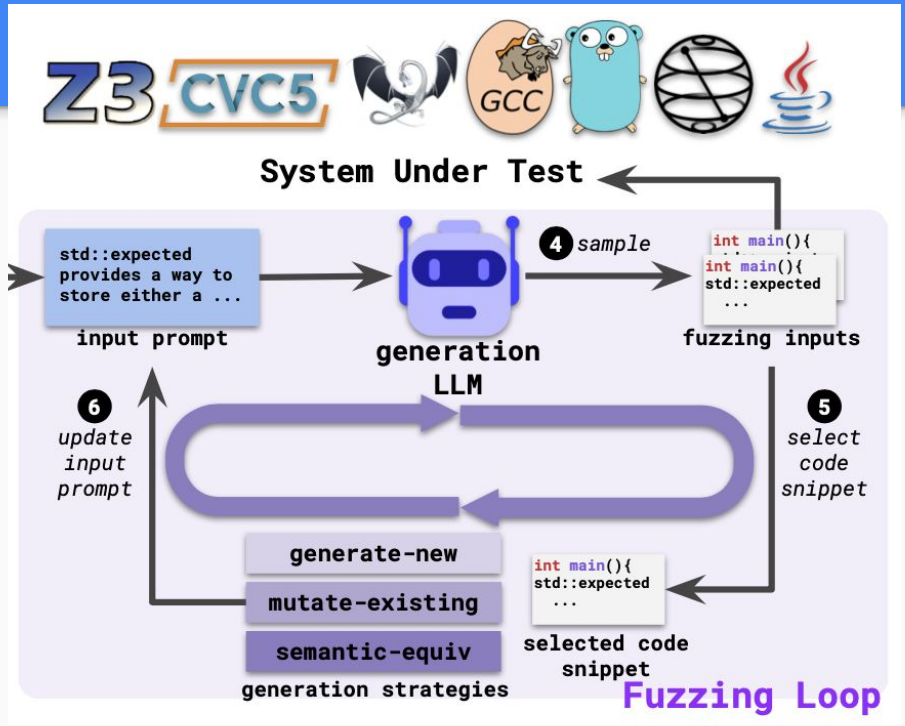
Part 1: Autoprompting

- Prompt engineering is hard!
- Large model distils natural-language documentation for fuzzer
- Evaluate candidate prompts over a few fuzzing runs
- Select prompt with best average validity of resulting code



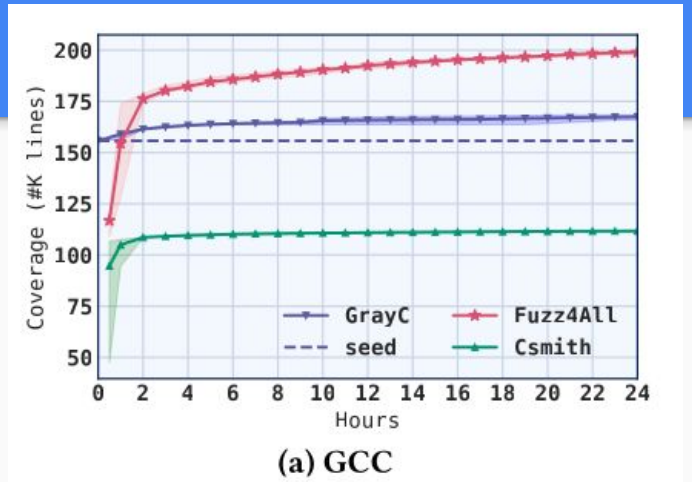
Part 2: Fuzzing Loop

- Take summary and best prompt from distiller
- One prompt → many diverse programs
- Small fuzzing LLM generates from scratch or mutates existing example
- Evaluate inputs against fuzzing oracle to find bugs



Fuzz4All Evaluation

- ~40% avg. line coverage improvement
- Overcomes plateaus encountered by traditional fuzzers
- Quality over quantity: fewer valid programs
- How long are these inputs?

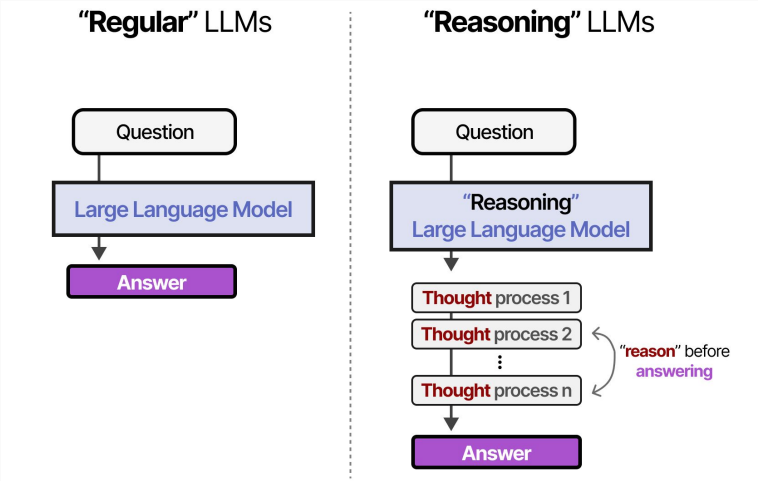


(a) GCC

Target	Fuzzer	# programs	% valid	Coverage	
GCC	GrayC	104,326	95.96%	167,453	
	Csmith	61,883	99.99%	111,668	
	Fuzz4All	44,324	37.26%	*198,927	+18.8%
G++	YARPGen	255,581	99.99%	166,614	
	Fuzz4All	26,365	40.74%	*210,743	+26.5%

My Project

- Extend Fuzz4All to work with newer models
- Trends: Reasoning, distilled, compact, open-weight, fine-tuning
- Compare coverage against models and parameter count
- Dive deeper into generated inputs



deepseek

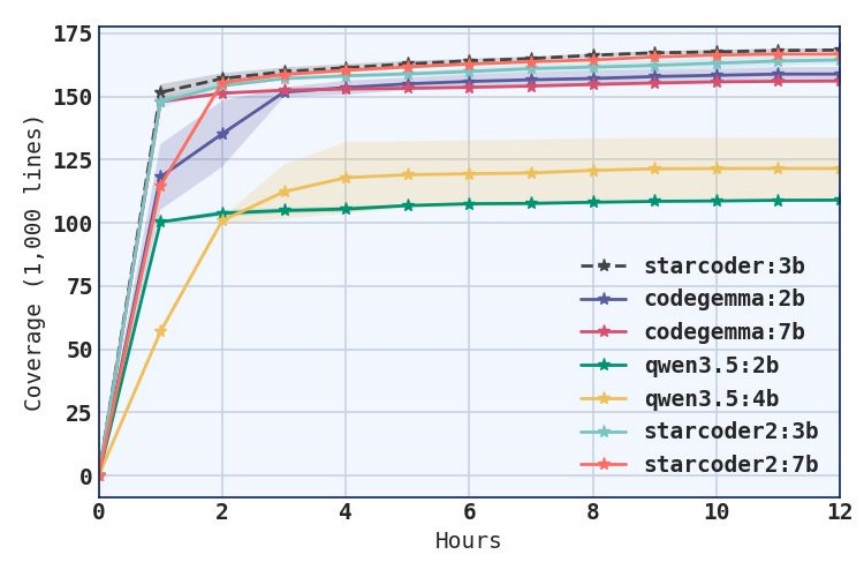
Experimental Setup

- Fuzz4All docker image on DADA (Ubuntu), no GPU
- Targets: GCC (C) and G++ (C++)
- Trials: 2 runs per target over 12 hrs
- Models: starcoder (2023, base), starcoder2 (2024), codegemma (2024), qwen3.5 (2026)
- Use mutate existing strategy and no autoprompting

Live Demo

Project Results

- No models beat the baseline!
- Larger models outperformed smaller ones
- Reasoning models (qwen3.5) performed the worst
- Sweet spot at 20 LoC per input and 50% valid programs



Analyzing Generated Inputs

- Several recurring mistakes
- Missing `#include` statements
- Using C++ features in C
- Undeclared variables and types
- EOF before main function completed (OOM)

Metric	Highest	Lowest
Coverage	starcoder	qwen3.5
# Programs	starcoder2	qwen3.5
% Valid	starcoder2	qwen3.5
Avg. LoC	codegemma	starcoder2

Conclusions

- Coding is not fuzzing!
- Generating inputs is open-ended and repetitive
- Simple & specialized beats latest & greatest
- Must balance quality and quantity
- Need a critical mass of input length and validity

Limitations and Next Steps

- Retry with more resources and for longer
- Generalize to other languages and targets
- Modify fuzzing prompt and strategy
- Play around with compiler options
- Compare LLM-generated inputs against traditional

End